

Tech Stack Selection & Decision Document

Hisab Kitab — Personal Finance Ledger System

Document Version: 1.0

Author: [Your Name]

Date: June 2025

1. Purpose of This Document

A Tech Stack Selection document is one of the most valuable artifacts you can show a recruiter. It proves that you did not just copy a tutorial stack — you evaluated alternatives, understood the tradeoffs, and made deliberate choices based on your project's specific constraints. Every decision below is justified against the system's requirements and your constraints (free deployment, solo developer, mobile-first).

2. Evaluation Criteria

Every technology was evaluated against five criteria. The first is **developer familiarity**, because a solo project built with unfamiliar tools ships late or not at all. The second is **deployment cost**, because the system must run entirely on free-tier cloud services. The third is **mobile-first compatibility**, because the primary user is on a smartphone. The fourth is **community and documentation quality**, because a solo developer needs strong self-service support. The fifth is **OOP and architectural alignment**, because the system is designed around a layered architecture and the developer prioritizes clean, object-oriented code.

3. Backend: FastAPI (Python)

Decision: FastAPI over Django REST Framework

FastAPI was chosen because it matches this project's needs more precisely than Django. The comparison is worth understanding in detail, because both are excellent choices in different contexts.

Django is an opinionated, batteries-included framework. It comes with an ORM, an admin panel, a templating engine, form handling, and session management. All of these are valuable when you are building a full-stack web application that renders HTML server-side. But Hisab Kitab is an API-only backend — the frontend is a separate React application. That means Django's

templating engine, form handling, admin panel, and session management are all dead weight. You would be carrying a framework optimized for a use case you are not serving.

FastAPI, by contrast, is optimized for building APIs. It uses Python type hints and Pydantic models to automatically validate request and response data — which maps perfectly to an OOP mindset, because every request and response becomes a typed Python class. It auto-generates interactive Swagger UI documentation at `/docs` with zero configuration, which is enormously useful for testing during development. It is async-native, which means it handles concurrent requests efficiently without blocking. And it is dramatically lighter than Django, which means faster cold starts on Render's free tier (where the server sleeps and wakes on demand).

The one advantage Django has over FastAPI in this context is its built-in ORM. However, this project uses SQLAlchemy separately, which is a more powerful and more Pythonic ORM than Django's, and it integrates cleanly with FastAPI.

Supporting libraries: SQLAlchemy 2.0 (async ORM), Pydantic v2 (data validation and schema definition), Alembic (database migrations), python-jose (JWT token handling), passlib[bcrypt] (password hashing), asyncpg (async PostgreSQL driver).

4. Database: PostgreSQL on Supabase

Decision: PostgreSQL over SQLite or MongoDB

SQLite was rejected because it is a file-based database unsuitable for server deployments where the filesystem is ephemeral. Render's free tier does not provide persistent disk storage, so any SQLite database would be wiped every time the server restarts. This makes SQLite viable only for local development.

MongoDB was rejected because this system has highly relational data. A Khata belongs to a User and contains many Entries. An Entry belongs to a Khata. A Party is linked to a Khata. These relationships are naturally expressed using foreign keys and JOIN operations — exactly what a relational database is optimized for. Using MongoDB would require manually managing these relationships in application code, which increases complexity without providing any benefit.

PostgreSQL was chosen because it is a production-grade relational database with strong support for decimal arithmetic (essential for financial calculations — floating point is never acceptable for money), excellent indexing for the date-range queries used in monthly reports, and full-text search capabilities for the global search feature.

Supabase was chosen as the hosting provider because it offers a free tier PostgreSQL instance with 500MB storage, automatic backups, a web-based table viewer, and a REST API that can be useful for debugging. Most importantly, it provides a standard PostgreSQL connection string that works identically with SQLAlchemy — there is no vendor lock-in.

5. Frontend: React with Vite (PWA)

Decision: React PWA over native Android or Flutter

A native Android application was rejected for two reasons: it requires a Play Store submission (which costs USD 25 and takes days to review), and maintaining a native Android codebase alongside a FastAPI backend as a solo developer doubles the technology surface area.

Flutter was rejected despite being a compelling cross-platform option because it would require learning Dart, a language outside the developer's existing skillset. The five-criteria evaluation makes this a straightforward rejection on the developer familiarity criterion.

React was chosen because the developer already has working knowledge of it. More importantly, a React application can be configured as a Progressive Web App — meaning the father can tap "Add to Home Screen" in Chrome and receive an app-like experience without visiting any app store. The PWA gets its own icon on the home screen, launches full-screen without the browser address bar, and works on any Android device running Chrome.

Vite was chosen over Create React App as the build tool because it is significantly faster for development (hot module replacement is near-instant), produces smaller production bundles, and has better first-class PWA support via the `vite-plugin-pwa` package.

Supporting libraries: Axios (API communication), React Query / TanStack Query (server state management and caching), React Router v6 (client-side routing), Zustand (lightweight client state management), date-fns (date arithmetic and formatting).

6. Deployment Architecture

Backend: Render (Free Tier)

Render was chosen over Railway and Fly.io because it has the most straightforward free tier for Python web services. Deployment is triggered automatically from a GitHub push, which means the developer never manually deploys — pushing to the `main` branch updates production. The limitation of Render's free tier is that the server sleeps after 15 minutes of inactivity and takes 30-50 seconds to wake up. This is documented as a known behavior and handled in the frontend with an appropriate loading state.

Frontend: Vercel (Free Tier)

Vercel is the natural deployment target for React applications. It provides zero-configuration deployments, automatic HTTPS, global CDN distribution (meaning the React app loads fast from Nepal without a dedicated regional server), and automatic preview deployments for every Git branch. The free tier has no meaningful limitations for a single-user application.

Supabase's free tier provides a PostgreSQL instance that pauses after 1 week of inactivity. This is managed by ensuring the application makes at least one database query per week — a simple scheduled health-check endpoint achieves this.

7. Architecture Pattern: Layered (N-Tier) Architecture

The backend follows a strict four-layer architecture. Understanding why each layer exists is as important as knowing that it does.

The **API Layer** (FastAPI route handlers) has exactly one job: receive an HTTP request, delegate to the service layer, and return an HTTP response. It knows nothing about business rules or database queries. This layer handles authentication middleware and input deserialization only.

The **Service Layer** contains all business logic. When an entry is added to a Khata, the service layer checks that the Khata is OPEN before delegating the write to the repository. When a Khata is settled, the service layer validates the state transition. This layer contains no SQL — it only calls repository methods.

The **Repository Layer** contains all database queries. A repository method like `get_outstanding_khatas(user_id)` knows exactly what SQL (via SQLAlchemy) to execute, but it knows nothing about why that query is being run. This separation means you can test the service layer with a mock repository and test the repository layer with a real database, independently.

The **Model Layer** (SQLAlchemy ORM models) defines the database schema as Python classes. These are the canonical source of truth for the data structure — Alembic reads these classes to generate migration files.

This architecture makes the codebase testable, maintainable by a single developer over time, and comprehensible to a technical interviewer reviewing it on GitHub.
